

PROJECT REPORT

Digital Locker System

Using DSCH Simulation Tool and Vivado Verilog with 4:1 Multiplexer

Subject	Digital Electronics & VLSI Design
Tool Used	DSCH (Digital Schematic) Simulation and Vivado Software
Key Component	4:1 Multiplexer, AND Gate, NOT Gate, LEDs
Passcode	1011 (A=1, B=0, C=1, D=1)

1. Abstract

The Digital Locker System is a simple digital electronics project designed using a 4:1 Multiplexer (MUX), logic gates, and LEDs simulated in the DSCH tool. The system grants access only when the correct 4-bit passcode (1011) is entered. Two input bits act as MUX select lines, while the remaining two bits are verified through an AND gate connected to MUX input I2. A match produces a HIGH output that activates the Unlock LED; all other combinations keep the Lock LED active.

2. Introduction

Digital security systems are integral to modern electronics, from smart lockers to access-control panels. This project implements a password-based locking mechanism using fundamental digital-logic building blocks, offering a practical introduction to multiplexer-driven design.

The primary objective is to demonstrate how a 4:1 MUX can replace complex combinational logic and serve as a compact decision element. The design was created and verified in DSCH, a widely-used academic tool for digital circuit schematic entry and simulation.

3. Components and Tools

3.1 Hardware Components

S.No	Component	Specification	Function
1	4:1 Multiplexer	2 select lines, 4 data inputs	Password selection logic
2	AND Gate (2-input)	CMOS logic	Verifies bits C and D
3	NOT Gate	CMOS logic	Drives Lock LED (inverted output)
4	Input Switches (x4)	A, B, C, D	Password bit entry
5	LEDs (x2)	Green / Red	Unlock / Lock indicator
6	Power Supply	VDD / GND	Circuit biasing

3.2 Software Tool

DSCH (Digital Schematic) is a free, open-source digital circuit simulator developed at the University of Bordeaux. It supports CMOS-level gate design, hierarchical schematics, and real-time logic simulation, making it ideal for educational projects.

4. Circuit Design Overview

4.1 Block Diagram Description

The overall system consists of three functional blocks:

- Input Block: Four binary switches (A, B, C, D) supply the password bits.
- Processing Block: A 4:1 MUX with select lines {A, B} and an AND gate on inputs C, D.
- Output Block: Unlock LED (active HIGH) and Lock LED (active LOW via NOT gate).

4.2 Pin Mapping

Switch	Connected To	Role	Correct Value
A	MUX Select S1	MSB of select	1
B	MUX Select S0	LSB of select	0
C	AND Gate in1 (I2 path)	Data verification	1
D	AND Gate in2 (I2 path)	Data verification	1

4.3 MUX Input Wiring

MUX Input	Connected To	Reason
I0	GND (logic 0)	Wrong select -> locked
I1	GND (logic 0)	Wrong select -> locked
I2	AND gate output	Correct select path (S1S0=10)
I3	GND (logic 0)	Wrong select -> locked

5. Working Principle

5.1 Step-by-Step Logic

1. Step 1: User sets switches A, B, C, D to a 4-bit combination.
2. Step 2: Bits A and B drive the MUX select lines S1 and S0.
3. Step 3: Only when A=1 and B=0 does the MUX route input I2 to the output.
4. Step 4: Bits C and D are ANDed together. The result feeds MUX input I2.
5. Step 5: MUX output = 1 only when {A,B}=10 AND C=1 AND D=1 (passcode 1011).
6. Step 6: Unlock LED turns ON; Lock LED turns OFF via the NOT gate.

5.2 Boolean Expression

$$\text{Unlock} = A \cdot \neg B \cdot C \cdot D$$

This expression is exactly what the circuit computes: A and NOT-B select input I2 of the MUX; the AND gate computes $C \cdot D$ which is fed into I2. The MUX therefore outputs $A \cdot \neg B \cdot C \cdot D$.

6. Truth Table (All 16 Combinations)

A	B	C	D	AND (C·D)	MUX Out	Unlock	Lock
0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	1
0	0	1	0	0	0	0	1
0	0	1	1	1	0	0	1
0	1	0	0	0	0	0	1
0	1	0	1	0	0	0	1
0	1	1	0	0	0	0	1
0	1	1	1	1	0	0	1
1	0	0	0	0	0	0	1
1	0	0	1	0	0	0	1
1	0	1	0	0	0	0	1
1	0	1	1	1	1	1	0
1	1	0	0	0	0	0	1
1	1	0	1	0	0	0	1
1	1	1	0	0	0	0	1
1	1	1	1	1	0	0	1

Note: Yellow row = correct passcode (1011) → only combination that activates Unlock.

7. Simulation Results

The circuit was designed and simulated in DSCH. The following sub-circuits were built and tested individually before integration:

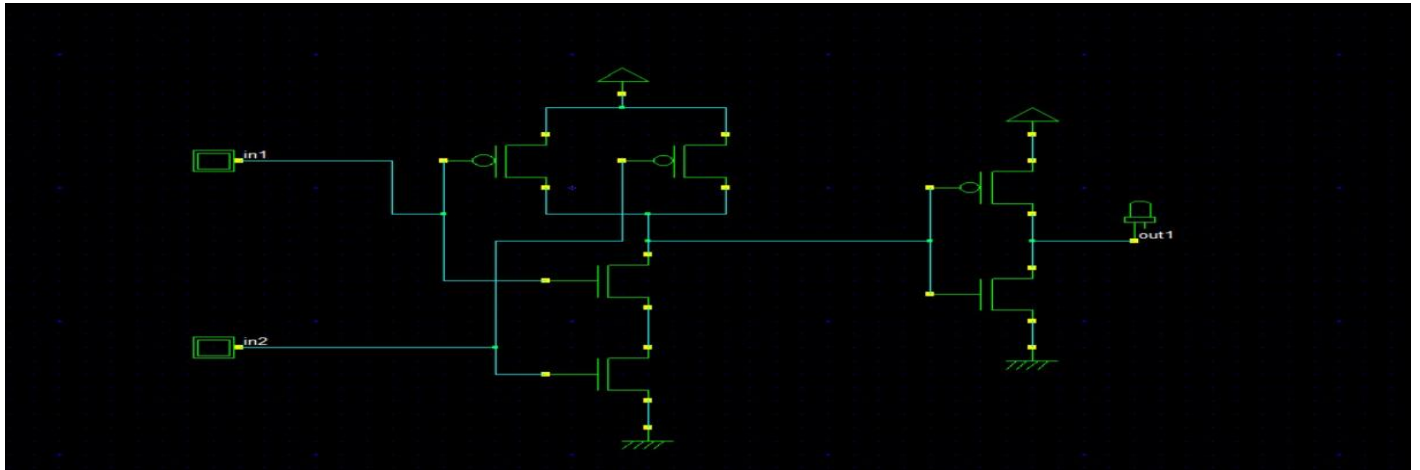


Fig.1: Design of CMOS AND gate circuit in DSCH

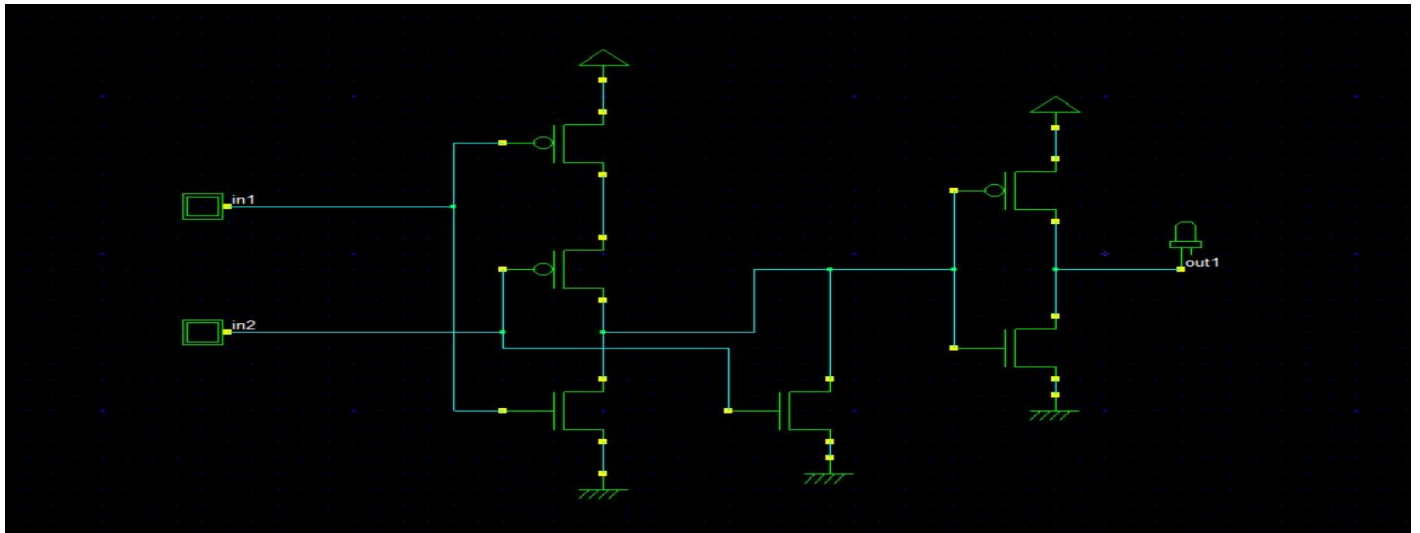


Fig.2: Design of CMOS OR gate circuit in DSCH

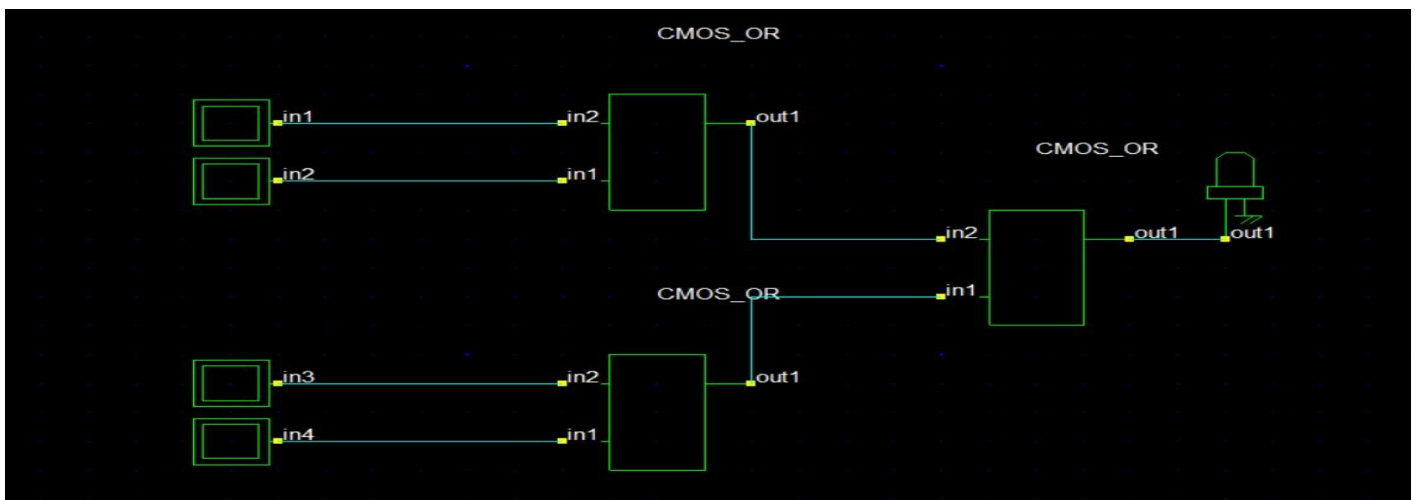


Fig.3: Design of CMOS 4-Input OR gate circuit in DSCH

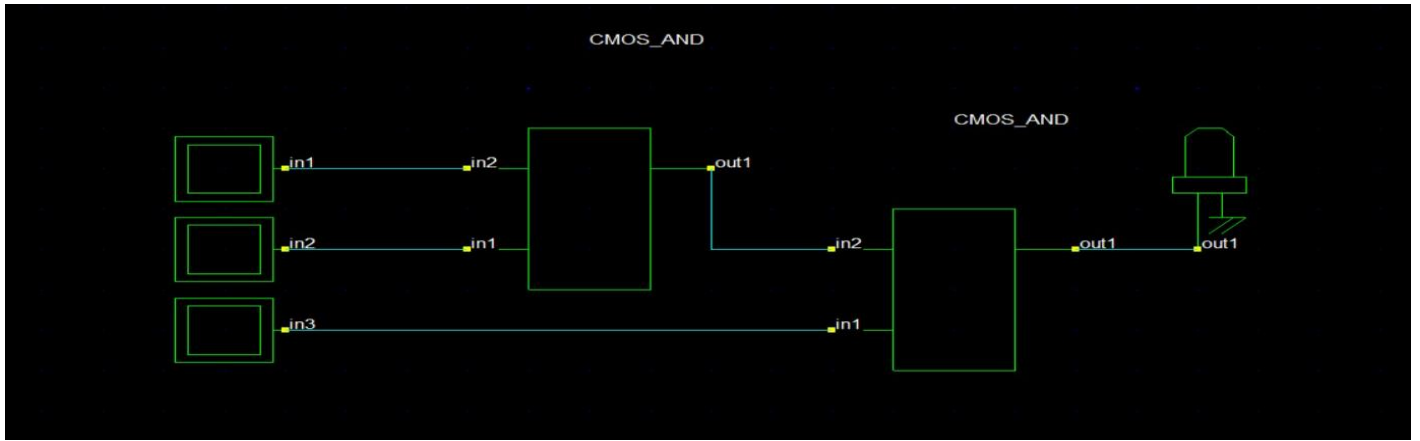


Fig.4: Design of CMOS 3-Input AND gate circuit in DSCH

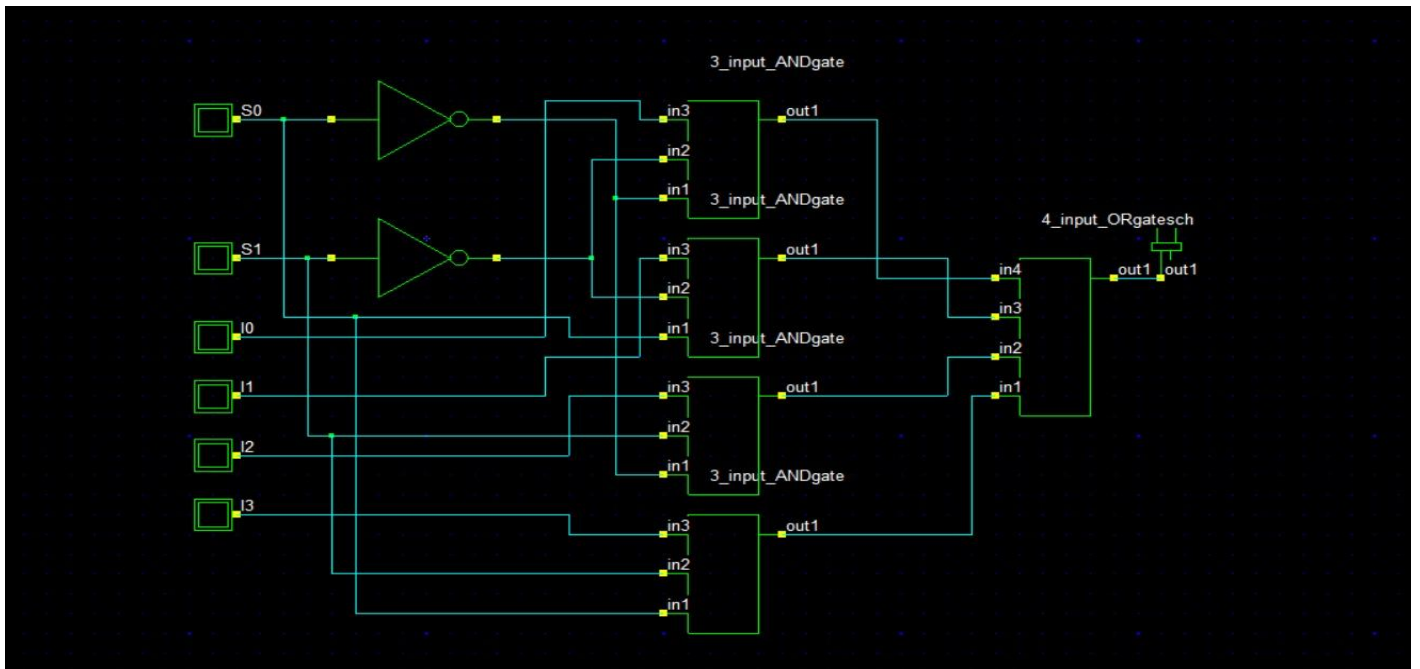


Fig.5: Design of 4:1 Multiplexer circuit in DSCH

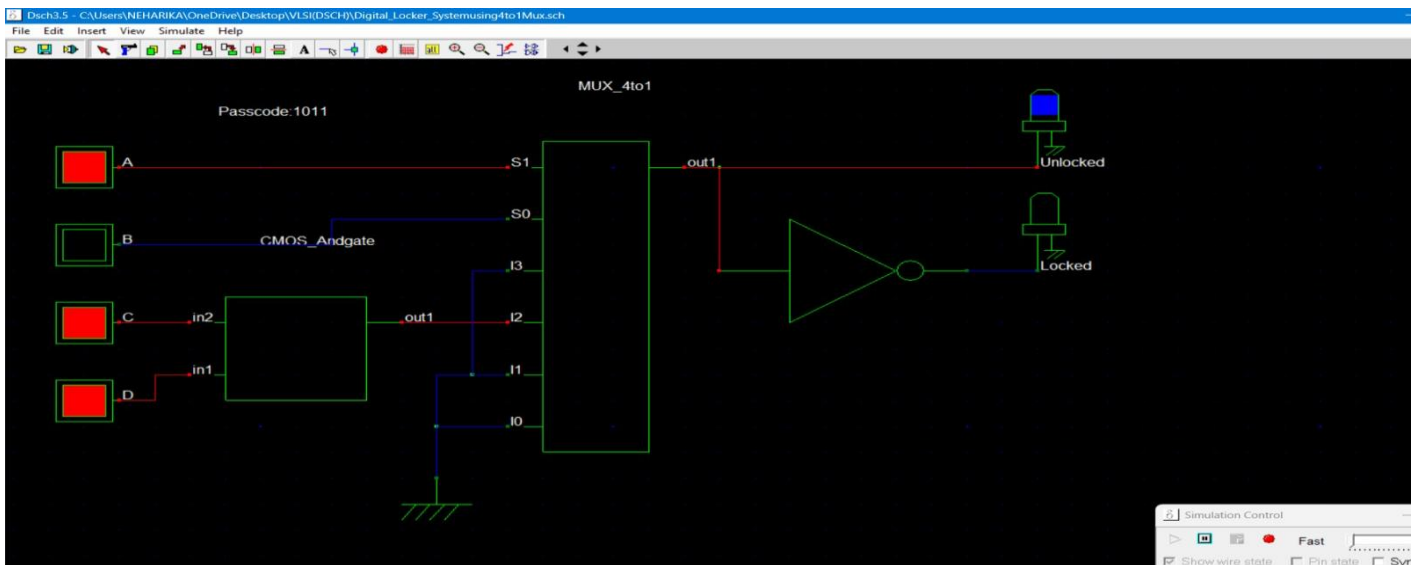


Fig.6: Design and simulated output of Proposed Digital Locker system using 4:1 mux

Fig.	Sub-circuit	Verification	Observation
1	CMOS AND Gate (2-input)	Pass	Output HIGH only when both inputs HIGH
2	CMOS OR Gate (2-input)	Pass	Output HIGH when any input HIGH
3	CMOS 4-input OR Gate	Pass	Correct sum-of-inputs behavior
4	CMOS 3-input AND Gate	Pass	Triple-input AND verified
5	CMOS 4:1 Multiplexer	Pass	Each select correctly routes I0-I3
6	Full Digital Locker System	Pass	Unlock at passcode 1011 only

In the final simulation (Fig. 6), setting A=1, B=0, C=1, D=1 produced Unlock=1 and Lock=0. All remaining 15 combinations kept Unlock=0, confirming the correct operation of the system.

8. Verilog RTL Implementation

A Register-Transfer Level (RTL) model was written in Verilog to validate the logic digitally and generate simulation waveforms using a testbench.

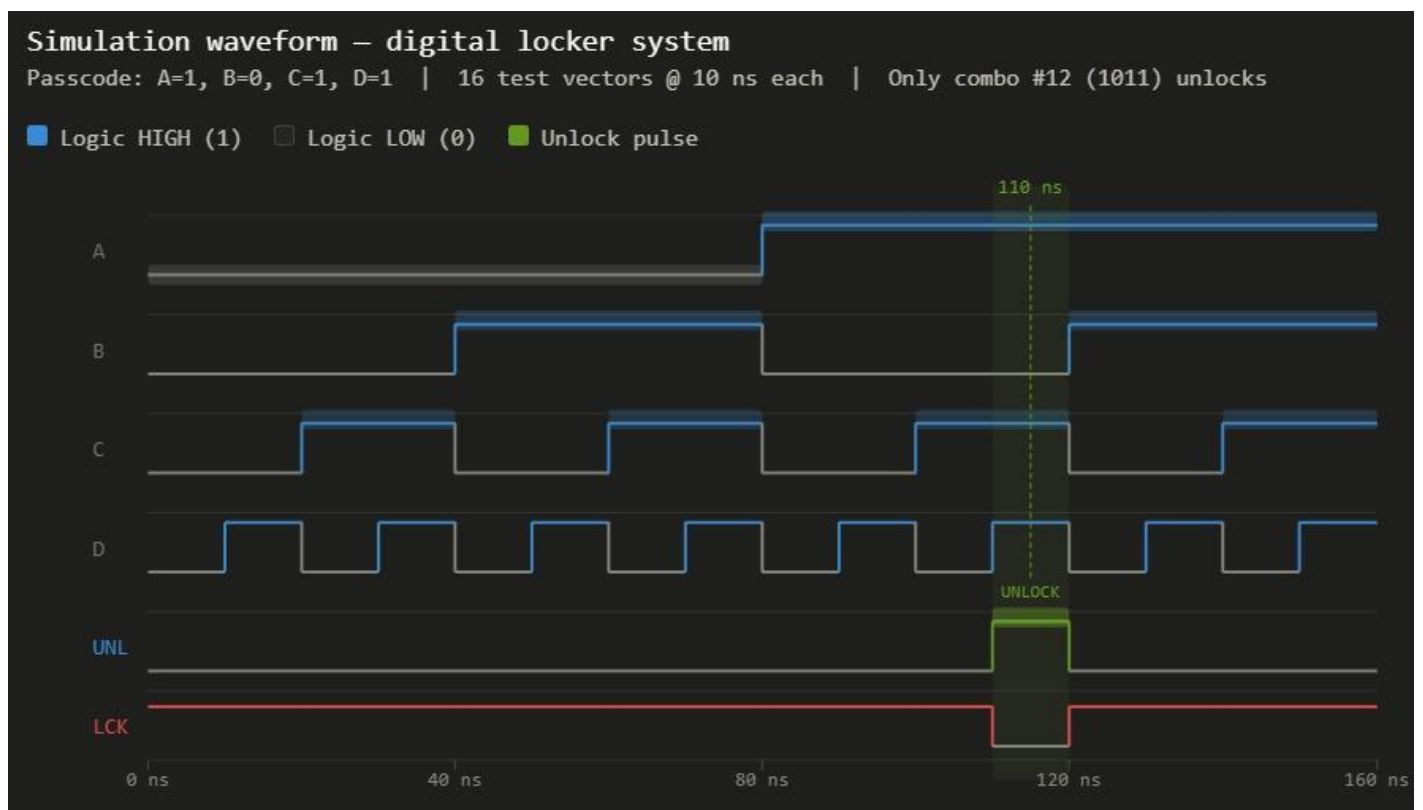


Fig.7: Expected Timing Waveforms

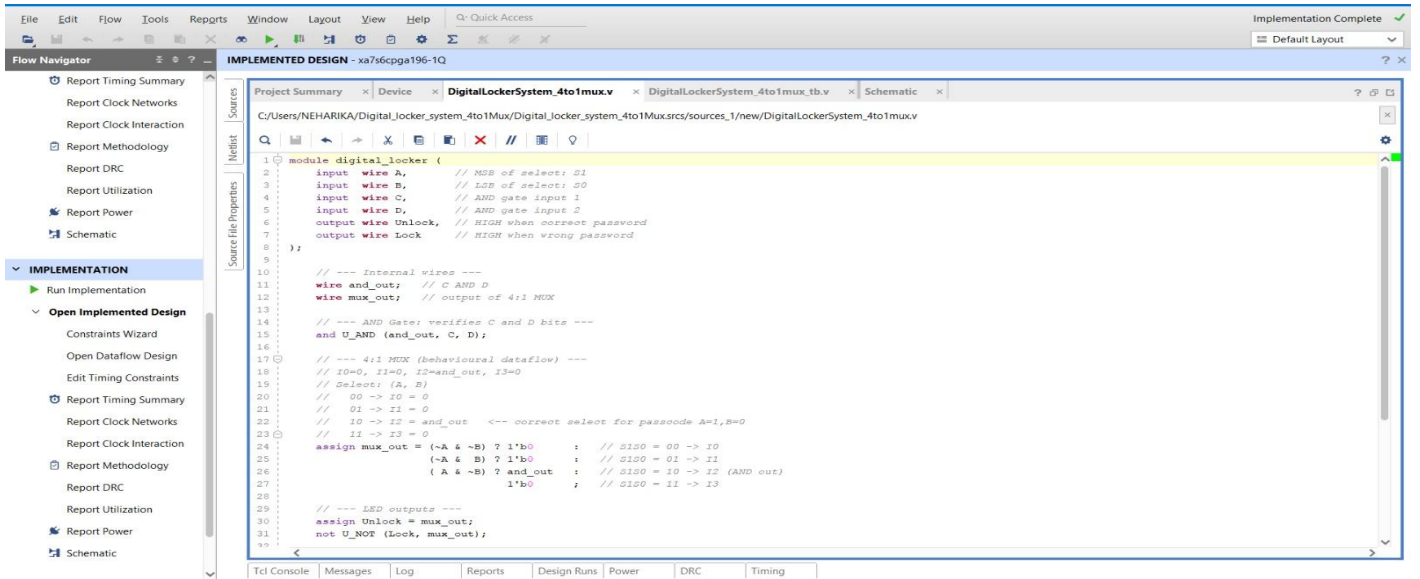


Fig.8: Verilog Design Code of Digital Locker System

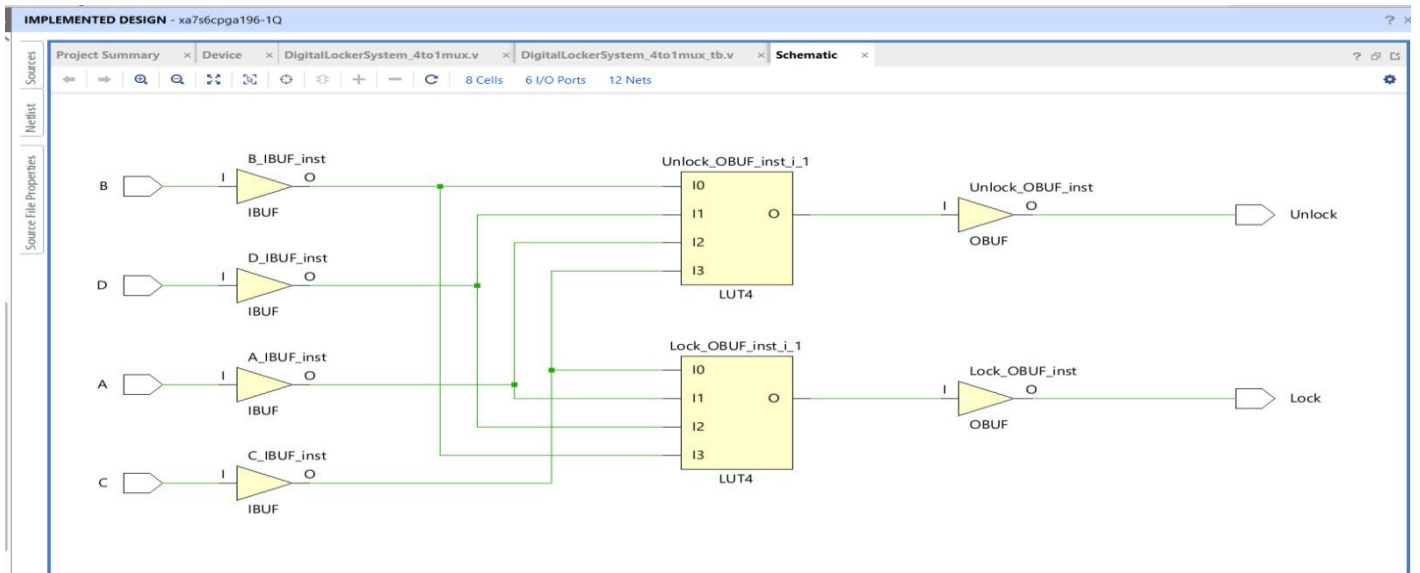


Fig.9: Netlist level Simulation Design of Digital Locker System

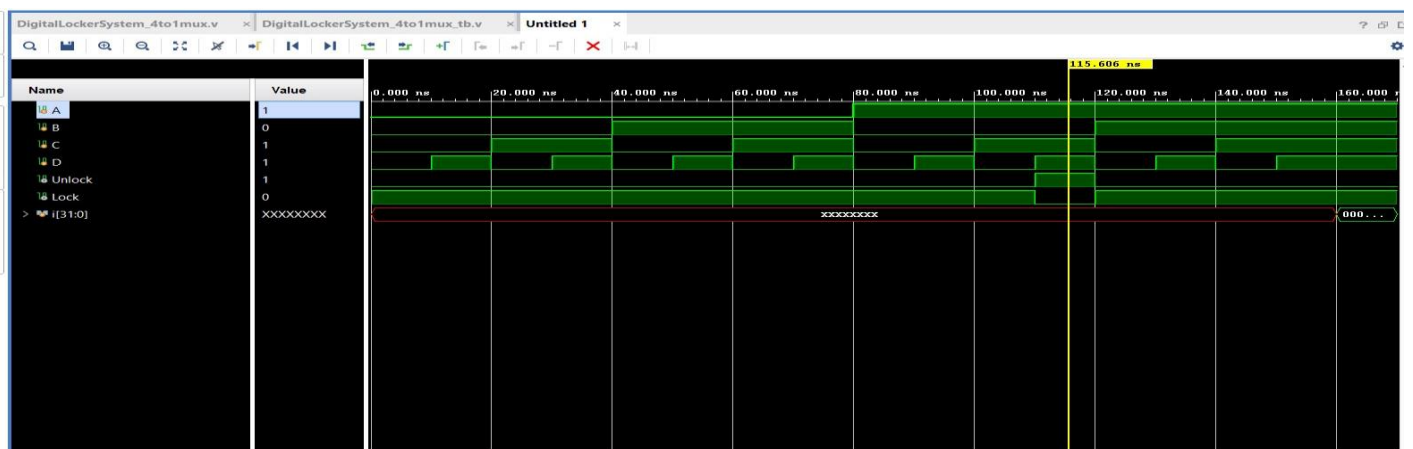


Fig.10: Simulated Waveforms of Digital Locker System

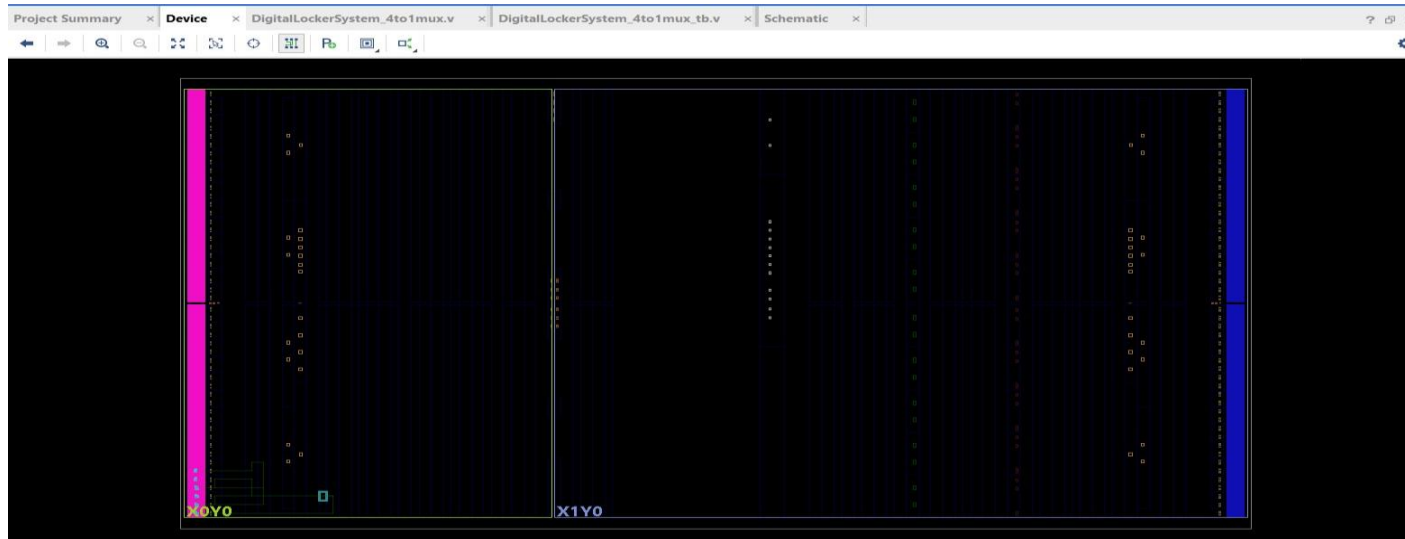


Fig.11: Layout level simulation of Digital Locker System

8.1 Design Module Summary

Port	Direction	Description
A, B	Input	MUX select lines (S1, S0)
C, D	Input	AND gate inputs (data bits)
Unlock	Output	HIGH when correct passcode entered
Lock	Output	Complement of Unlock (active otherwise)

8.2 Testbench Strategy

- All 16 input combinations applied sequentially at 10 ns intervals.
- VCD waveform dump generated for GTKWave visualization.
- Console output confirms Unlock=1 exclusively at combination 1011.

9. Synthesis & Netlist

To obtain the gate-level netlist diagram, the Verilog design is synthesized using:

Tool	Target Device	How to view netlist
Xilinx Vivado	Basys 3 (xc7a35tcbg236-1)	Open Synthesized Design → Tools → Schematic
Xilinx ISE	Spartan-3E (xc3s100e)	Synthesize-XST → View Technology Schematic
Intel Quartus	Cyclone II (EP2C5T144C8)	Compile → Tools → Netlist Viewers → RTL Viewer

10. Conclusion

The Digital Locker System was successfully designed, simulated in DSCH, and validated in Verilog. The project demonstrated how a 4:1 MUX and basic logic gates can implement a compact password-verification circuit using only four binary input switches.

Key concepts reinforced through this project:

- Multiplexer operation and select-line based data routing.
- CMOS gate design and transistor-level simulation in DSCH.
- Boolean logic minimisation and direct hardware mapping.
- Verilog HDL design flow from RTL coding to waveform verification.
- Synthesis and netlist generation for FPGA target devices.

This design can be extended by increasing the passcode length (more input bits and a wider MUX), adding a counter to lock out after repeated wrong attempts, or integrating with a physical locking mechanism via a relay driver circuit.

11. References

7. Morris Mano, Michael Ciletti — Digital Design With an Introduction to the Verilog HDL, 5th Edition, Pearson.
8. Samir Palnitkar — Verilog HDL: A Guide to Digital Design and Synthesis, 2nd Edition, Prentice Hall.
9. DSCH Documentation — University of Bordeaux, Microelectronics Group, <https://www.microwind.org>.
10. Xilinx Vivado Design Suite User Guide — UG910, Xilinx Inc., 2023.

12. Design Source Code

```
module digital_locker (  
  
    input wire A,    // MSB of select: S1  
  
    input wire B,    // LSB of select: S0  
  
    input wire C,    // AND gate input 1  
  
    input wire D,    // AND gate input 2  
  
    output wire Unlock, // HIGH when correct password  
  
    output wire Lock   // HIGH when wrong password  
  
);  
  
    // --- Internal wires ---  
  
    wire and_out; // C AND D  
  
    wire mux_out; // output of 4:1 MUX  
  
    // --- AND Gate: verifies C and D bits ---  
  
    and U_AND (and_out, C, D);  
  
    // --- 4:1 MUX (behavioural dataflow) ---  
  
    // I0=0, I1=0, I2=and_out, I3=0
```

```

// Select: {A, B}

// 00 -> I0 = 0

// 01 -> I1 = 0

// 10 -> I2 = and_out <-- correct select for passcode A=1,B=0

// 11 -> I3 = 0

assign mux_out = (~A & ~B) ? 1'b0 : // S1S0 = 00 -> I0

               (~A & B) ? 1'b0 : // S1S0 = 01 -> I1

               ( A & ~B) ? and_out : // S1S0 = 10 -> I2 (AND out)

               1'b0 ; // S1S0 = 11 -> I3

// --- LED outputs ---

assign Unlock = mux_out;

not U_NOT (Lock, mux_out);

endmodule

```

13. Design Source Code

```

`timescale 1ns/1ps

module tb_digital_locker;

// --- DUT inputs (reg) ---

reg A, B, C, D;

// --- DUT outputs (wire) ---

wire Unlock, Lock;

// --- Instantiate Device Under Test ---

digital_locker DUT (

    .A(A), .B(B), .C(C), .D(D),

    .Unlock(Unlock), .Lock(Lock)

);

// --- VCD dump for waveform viewer (GTKWave etc.) ---

initial begin

```

```

$dumpfile("digital_locker_tb.vcd");

$dumpvars(0, tb_digital_locker);

end

// --- Stimulus: apply all 16 combinations ---

integer i;

initial begin

    $display("=====");

    $display(" Digital Locker System - Full Truth Table");

    $display(" Correct Passcode: A=1 B=0 C=1 D=1");

    $display("=====");

    $display(" Time | A B C D | Unlock | Lock | Status");

    $display("-----");

    for (i = 0; i < 16; i = i + 1) begin

        {A, B, C, D} = i[3:0];

        #10;

        $display(" %3t | %b %b %b %b | %b | %b | %s",

            $time, A, B, C, D, Unlock, Lock,

            (Unlock == 1'b1) ? "*** UNLOCKED ***" : "LOCKED");

    end

    $display("=====");

    #10;

    $finish;

end

endmodule

```